BCSCHOOL

Empowering Digital Skills For The Jobs Of The Future

by

BC Soft
consulenza sistemi informativi

AiRED
Artificial Intelligence Ricerca e Sviluppo

JAVA JEE
-
Incapsulamento e visibilità

TEACHER

Vincenza Ruvolo

vincenza.ruvolo@bcsoft.net

Sommario

- I paradigmi della programmazione ad oggetti
- Incapsulamento
- Quando utilizzare l'incapsulamento
- Come usare l'incapsulamento
- Gestione dei package
- Modificatori d'accesso
- Il modificatore static

I paradigmi della programmazione ad oggetti

Ciò che caratterizza un linguaggio orientato agli oggetti è il supporto che esso offre ai cosiddetti paradigmi della programmazione ad oggetti:

- Incapsulamento
- Ereditarietà
- Polimorfismo

Astrazione

L’astrazione potrebbe definirsi come “l’arte di sapersi concentrare solo sui dettagli veramente essenziali nella descrizione di un’entità”.

Astrazione

Nella programmazione ad oggetti si hanno tre livelli di astrazione:

- **astrazione funzionale**
- **astrazione dei dati**
- **astrazione del sistema**

Astrazione

Si utilizza **l'astrazione funzionale** ogni volta che viene implementato un metodo. Infatti, tramite un metodo, si riesce a portare all'interno di un'applicazione un concetto dinamico, sinonimo di azione, funzione.

Astrazione

Si adopera **l'astrazione dei dati** ogni volta che viene definita una classe, raccogliendo in essa solo le caratteristiche e le funzionalità essenziali degli oggetti che essa deve definire nel contesto in cui ci si trova.

Riuso

Il riuso è da considerarsi una conseguenza dell’astrazione e degli altri paradigmi della programmazione ad oggetti, in quanto spesso si prende spunto da un codice e lo si **customizza.**

Riuso

Naturalmente anche in questo caso si possono avere tre livelli:

- Riuso funzionale.
- Riuso dei dati.
- Riuso concettuale.

Incapsulamento

Attraverso l’incapsulamento una classe riesce ad acquisire caratteristiche di **robustezza, indipendenza e riusabilità**.

Incapsulamento

Si supponga, a scopo illustrativo, di voler scrivere un'applicazione che utilizza la seguente classe, la quale astrae in maniera semplice il concetto di data:

```
public class MyData {

    public int giorno;
    public int mese;
    public int anno;

}
```

Esempio

```
1  package Slide_6;
2
3  public class TestMyData {
4
5      public static void main(String[] args) {
6
7          //istanzio oggetto classe MyData
8
9          MyData oggettoData = new MyData();
10
11         oggettoData.
12
13     }
14
15 }
16
```

anno : int - MyData
giorno : int - MyData
mese : int - MyData
equals(Object obj) : boolean - Object
getClass() : Class<?> - Object
hashCode() : int - Object
notify() : void - Object
notifyAll() : void - Object
toString() : String - Object
wait() : void - Object
wait(long timeoutMillis) : void - Object

Press '^Space' to show Template Proposals

Incapsulamento

Ogni volta che serve un oggetto MyData, il codice da scrivere nel main sarà simile al seguente:

```
...
MyData unaData = new MyData();
unaData.giorno = 14;
unaData.mese = 4;
unaData.anno = 2004;
...
```

Esempio

```java
package Slide_6;

import java.util.Scanner;

public class TestMyData {

    public static void main(String[] args) {

        //istanzio oggetto classe MyData

        MyData ogData = new MyData();
        System.out.println("Oggi è ");
        ogData.anno=2020;
        ogData.mese= 5;
        ogData.giorno=30;

        System.out.println("L'anno dichiarato è: " + ogData.anno );
        System.out.println("Il giorno dichiarato è: " + ogData.giorno );
        System.out.println("Il mese dichiarato è: " + ogData.mese );


        System.out.println("************************");

        MyData ogData2 = new MyData();

        Scanner input= new Scanner(System.in);
        System.out.println("Inserisci il tuo anno di nascita ");
        ogData2.anno= input.nextInt();
        System.out.println("Inserisci il tuo mese di nascita ");
        ogData2.mese= input.nextInt();
        System.out.println("Inserisci il tuo giorno di nascita ");
        ogData2.giorno=input.nextInt();

        System.out.println("L'anno dichiarato è: " + ogData2.anno + " \nmese: " + ogData2.mese + "\ngiorr

    }

}
```

```java
package Slide_6;

//definisco classe MyData

public class MyData {

    //definisco attributi
    public int giorno;
    public int mese;
    public int anno;
```

Incapsulamento

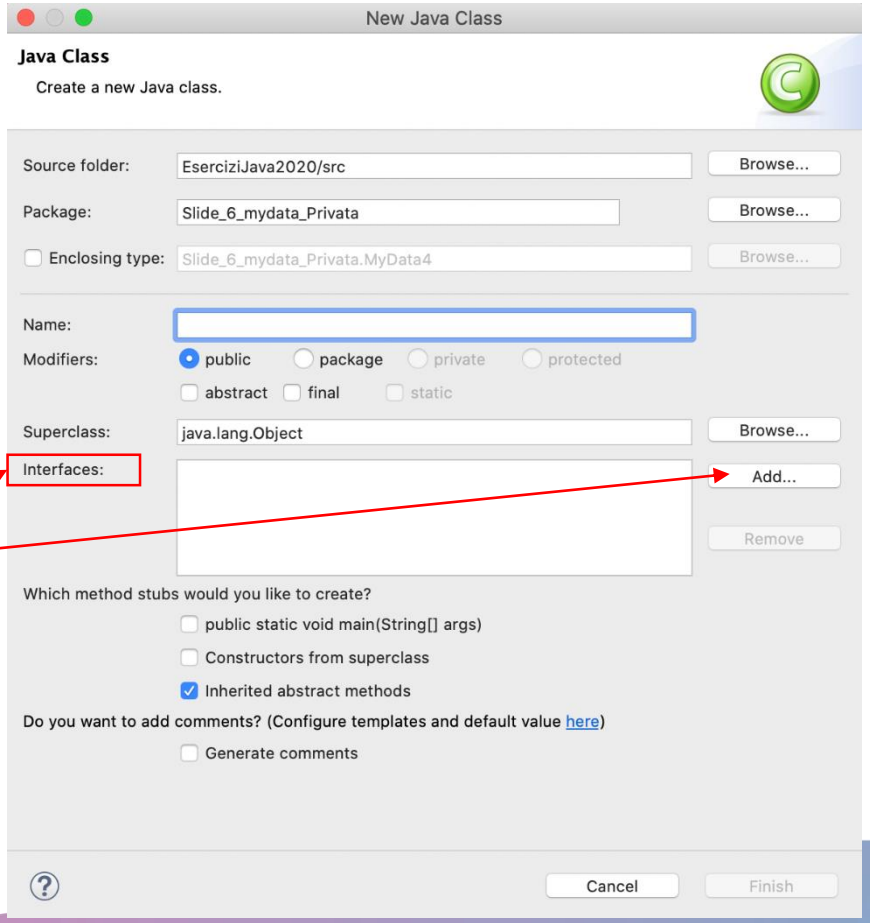
Si supponga che l'applicazione permetta all'utente di inserire la sua data di nascita, magari mediante un'interfaccia grafica. In tal caso il codice da scrivere sarà simile al seguente:

```
...
MyData unaData = new MyData();
unaData.giorno = interfaccia.dammiGiornoInserito();
unaData.mese = interfaccia.dammiMeseInserito();
unaData.anno = interfaccia.dammiAnnoInserito();
...
```

Esempio

MI CREA IN AUTOMATICO GLI OVERRIDE DEI METODI DELL'INTERFACCIA

CREO CLASSE COME **IMPLEMENTAZIONE** DELL'INTERFACCIA!



Esempio

```java
package Slide_6_mydata;

import java.util.Scanner;


public class MyData3 implements InterfacciaGiorno {

    //definisco attributi
        public int giorno;
        public int mese;
        public int anno;

    @Override
    public int dammiGiornoInserito() {
        //codice del metodo personalizzato presernte in interfaccia

        //in questo caso chiedo di inserire un numero da tastiera
        Scanner input= new Scanner(System.in);
        System.out.println("Inserisci il tuo anno di nascita ");

        //il numero inserito lo associo all'attributo giorno
        this.giorno= input.nextInt();

        /*N.B. di norma quando istanziamo un oggetto di una classe utilizziamo il nome dell'oggetto
        e il metodo applicato come in questo caso:

        MyData ogData2 = new MyData();
        Scanner input= new Scanner(System.in);
        System.out.println("Inserisci il tuo giorno di nascita ");
        ogData2.giorno=input.nextInt();
        */


        //richiedo c he mi deve essere restituito il giorno
        return this.giorno;
    }

    @Override
    public int dammiMeseInserito() {
        // TODO Auto-generated method stub
        return this.mese;
    }
```

```java
package Slide_6_mydata;

public interface InterfacciaGiorno {

    public int dammiGiornoInserito();
    public int dammiMeseInserito();
    public int dammiAnnoInserito();


}
```

Esempio

Istanzio oggetto appartenente alla classe

MyData3 unaData = **new** MyData3();

Utilizzo il metodo .dammiGiornoInserito() (metodo customizzato nella classe appartenente all'interfaccia) applicandolo attributo giorno (attributo della classe MyData3)

unaData.giorno =unaData.dammiGiornoInserito();

Esempio

```java
package Slide_6_mydata;

public class TestMyData3 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        //Istanzio oggetto appartenente alla classe

        MyData3 unaData = new MyData3();

        //Utilizzo il metodo .dammiGiornoInserito() (metodo castomizzato
        //nella classe appartenente all'interfaccia) applicandolo attributo giorno (attributo della class
        unaData.giorno =unaData.dammiGiornoInserito();

    }

}
```

Incapsulamento

I metodi **dammiGiornoInserito(), dammiMeseInserito() e dammiAnnoInserito()** dell'oggetto interfaccia restituiscono un intero inserito dall'utente mediante l'applicazione.

```
package mydata;

public interface Interfaccia {

	public int dammiGiornoInserito();
	public int dammiMeseInserito();
	public int dammiAnnoInserito();
}
```

Incapsulamento

La classe MyData è così implementata:

```
package mydata;

public class MyData implements Interfaccia {

	public int giorno;
	public int mese;
	public int anno;
	public int dammiGiornoInserito(){return this.giorno;}
	public int dammiMeseInserito(){return this.mese;}
	public int dammiAnnoInserito(){return this.anno;}
}
```

Incapsulamento

Si supponga che l’utente abbia inserito rispettivamente i valori 32 per il giorno, 13 per il mese e 1800 per l’anno.

Incapsulamento

Correzione degli errori:

1. Si potrebbe limitare la possibilità degli inserimenti sull’interfaccia grafica all’utente.

Incapsulamento

2. Potremmo delegare al codice dei metodi dammiGiornoInserito(), dammiMeseInserito() e dammiAnnoInserito() dell’oggetto interfaccia i controlli necessari alla giusta impostazione della data.

Incapsulamento

Utilizzare l’incapsulamento modificando la classe MyData nel modo seguente:

Esempio

```java
1  package Slide_6_mydata_Privata;
2
3  public class MyData4 {
4
5      //dichiaro attributi classe
6        private int giorno;
7        private int mese;
8        private int anno;
9
10
```

```java
public class TestMyData4 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        MyData4 ogData2 = new MyData4();

        ogData2.
    }

}
```

- equals(Object obj) : boolean - Object
- getClass() : Class<?> - Object
- hashCode() : int - Object
- notify() : void - Object
- notifyAll() : void - Object
- toString() : String - Object
- wait() : void - Object
- wait(long timeoutMillis) : void - Object
- wait(long timeoutMillis, int nanos) : void - Object

Avendo dichiarato privati gli attributi della classe (MyData4) istanziando un nuovo oggetto della classe MyData4 non posso utilizzare gli attributi sull'oggetto con l'operatore dot(.)

Esempio

Devo creare metodi Get e Set sugli attributi privati definiti nella classe MyData4!

```
1  package Slide_6_mydata_Privata;
2
3  public class MyData4 {
4
5      //dichiaro attributi classe
6        private int giorno;
7        private int mese;
8        private int anno;
9
10
11 }
12
```

Console | Problems | Debug Shell

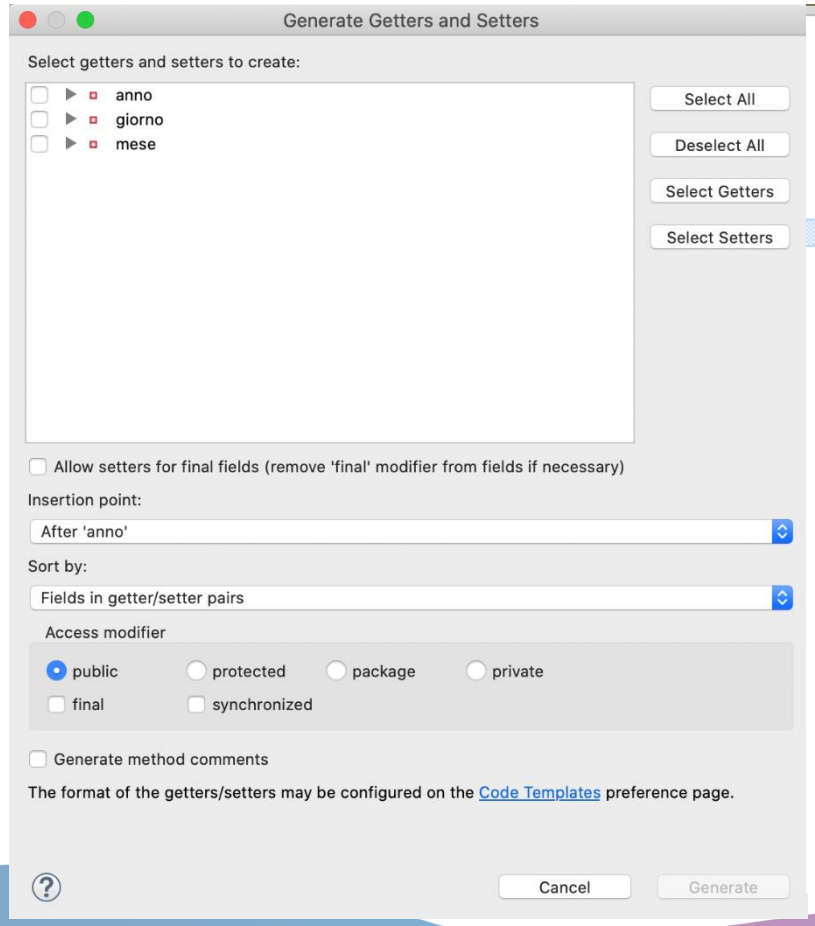
TestMyData3 [Java Application] /Library/Java/Java

Inserisci il tuo anno di nascita

- Undo Typing ⌘Z
- Revert File
- Save ⌘S
- Open Declaration F3
- Open Type Hierarchy F4
- Open Call Hierarchy ^⌥H
- Show in Breadcrumb ⌥⌘B
- Quick Outline ⌘O
- Quick Type Hierarchy ⌘T
- Open With ▶
- Show In ⌥⌘W ▶
- Cut ⌘X
- Copy ⌘C
- Copy Qualified Name
- Paste ⌘V
- Quick Fix ⌘1
- Source ⌥⌘S ▶
- Refactor ⌥⌘T ▶
- Local History ▶
- References ▶
- Declarations ▶
- Add to Snippets...
- Run As ▶
- Debug As ▶
- Profile As ▶
- Team ▶
- Compare With ▶
- Replace With ▶

- Toggle Comment ⌘7
- Remove Block Comment ^⌘\
- Generate Element Comment ⌥⌘J
- Correct Indentation ⌘I
- Format ⇧⌘F
- Format Element
- Add Import ⇧⌘M
- Organize Imports ⇧⌘O
- Sort Members...
- Clean Up...
- Override/Implement Methods...
- Generate Getters and Setters...
- Generate Delegate Methods...

Esempio




```java
1  package Slide_6_mydata_Privata;
2
3  public class MyData4 {
4
5      //dichiaro attributi classe
6        private int giorno;
7        private int mese;
8        private int anno;
9      public int getGiorno() {
10         return giorno;
11     }
12     public void setGiorno(int giorno) {
13         this.giorno = giorno;
14     }
15     public int getMese() {
16         return mese;
17     }
18     public void setMese(int mese) {
19         this.mese = mese;
20     }
21     public int getAnno() {
22         return anno;
23     }
24     public void setAnno(int anno) {
25         this.anno = anno;
26     }
27
28
29 }
30
```

Esempio

```
4
5  public class TestMyData4 {
6
7      public static void main(String[] args) {
8          // TODO Auto-generated method stub
9
10         MyData4 ogData2 = new MyData4();
11
12         ogData2.
13     }
14
15 }
16
```

- equals(Object obj) : boolean - Object
- getAnno() : int - MyData4
- getClass() : Class<?> - Object
- getGiorno() : int - MyData4
- getMese() : int - MyData4
- hashCode() : int - Object
- notify() : void - Object
- notifyAll() : void - Object
- setAnno(int anno) : void - MyData4
- setGiorno(int giorno) : void - MyData4
- setMese(int mese) : void - MyData4

Press '^Space' to show Template Proposa

Incapsulamento

```
package mydata;
public class MyData {
   private int giorno;
   private int mese;
   private int anno;
   public void setGiorno(int g) {
     if (g > 0 && g <= 31) {
         giorno = g;
     } else {
         System.out.println("Giorno non valido");
     }
   }

   public int getGiorno() {
      return giorno;
   }

   public void setMese(int m) {
      if (m > 0 && m <= 12) {
         mese = m;
      } else {
         System.out.println("Mese non valido");
      }
   }

   public int getMese() {
         return mese;
   }
   public void setAnno(int a) {
         anno = a;
   }
   public int getAnno() {
      return anno;
   }
}
```

Esempio

```java
package Slide_6_mydata_Privata;



public class TestMyData4 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        //Istanzio oggetto della classe MyData4

        MyData4 ogData2 = new MyData4();

        //Utilizzo metodo set circa il giorno per definire il giorno

        ogData2.setGiorno(4);

        //Utilizzo metodo get per farmi restituire nella stampa il giorno, definito col metodo set

        System.out.println("Il giorno è: " + ogData2.getGiorno());

    }

}
```

Console | Problems | Debug Shell | Servers

```
<terminated> TestMyData4 [Java Application] /Library/Java/JavaVirtualMachines/jdk-14.0.1.jdk/Contents/Home/bin/java  (30 mag 2020, 18:09:12
Il giorno è: 4
```

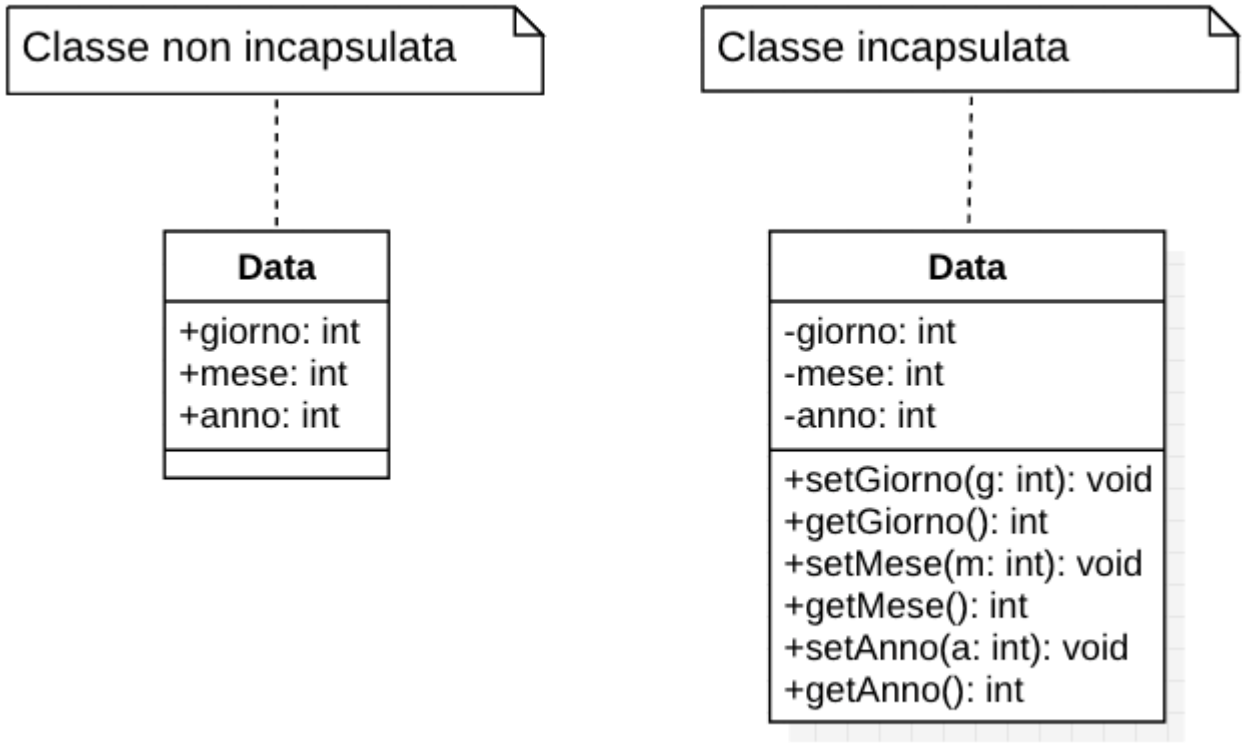
Incapsulamento

L’accesso ai dati privati potrà essere fornito da un’interfaccia pubblica costituita da metodi dichiarati public e quindi accessibili da altre classi.

Incapsulamento

Implementare l’incapsulamento con codice Java consiste il più delle volte nel dichiarare i dati privati e fornire alla classe metodi pubblici di tipo **set** e **get** per accedervi rispettivamente in scrittura e lettura.

Incapsulamento



Incapsulamento

È importante l'utilizzo dell'incapsulamento poiché avendo dichiarato i dati privati, per la definizione del modificatore ***private*** essi non saranno più accessibili mediante l'operatore **dot (.),** a meno che il codice che vuole accedere al dato privato non si trovi nella classe che lo ha dichiarato.

Incapsulamento

Visto che i codici iniziali produrrebbero un errore in compilazione in quanto tenterebbero di assegnare valori a variabili non visibili in quel contesto (classi diverse dalla classe Data), bisogna sostituirli con i seguenti:

Incapsulamento

```
...
MyData unaData = new MyData();
unaData.setGiorno(14);
unaData.setMese(4);
unaData.setAnno(2004);
...
```

```
...
MyData unaData = new MyData();
Interfaccia interfaccia = unaData;
unaData.setGiorno(interfaccia.dammiGiornoInserito());
unaData.setMese(interfaccia.dammiMeseInserito());
unaData.setAnno(interfaccia.dammiAnnoInserito());
...
```

Incapsulamento

I metodi set e get hanno implementazioni che si trovano internamente alla classe MyData e quindi possono accedere ai dati privati.

Incapsulamento

Se il metodo setGiorno(), viene cambiato nel modo seguente:

```
public void setGiorno(int g) {
  if (g > 0 && g <= 31 && mese != 4) {
    giorno = g;
  }
  else {
    System.out.println("Giorno non valido");
  }
}
```

Incapsulamento

Bisogna riconsiderare la classe MyData, avendo cambiato una parte di codice.

Incapsulamento

I **metodi set** sono detti **mutator methods**, ovvero metodi che mutano (il valore della variabile), mentre i **metodi get** sono detti **accessor methods** ovvero metodi d'accesso.

Incapsulamento

Si supponga il seguente codice:

```
...
MyData unaData = new MyData();
Interfaccia interfaccia = unaData;
unaData.setGiorno(interfaccia.dammiGiornoInserito());
unaData.setMese(interfaccia.dammiMeseInserito());
unaData.setAnno(interfaccia.dammiAnnoInserito());
...
```

Incapsulamento

Si supponga di volere aggiungere la verifica dell’effettivo successo dell’impostazione dei dati dell’oggetto unaData, stampandone i dati a video:

Incapsulamento

L’istruzione:

System.out.println(unaData.giorno);

restituirà un errore in compilazione.

Incapsulamento

L'istruzione:

System.out.println(unaData.setGiorno());

<u>non ha senso</u> perché il tipo di ritorno del metodo setGiorno() è void.

Incapsulamento

L’istruzione

System.out.println(unaData.getGiorno());

è l’unica soluzione rimanente, ed è quella corretta.

Incapsulamento

Nella seguente classe l’accessor method gestisce l’accesso ad un conto bancario personale, mediante l’inserimento di un codice segreto:

Incapsulamento

```
public class ContoBancario {
  private String contoBancario="5000000 Euro";
   private int codice = 1234;
  private int codiceInserito;

  public void setCodiceInserito(int cod) {
     codiceInserito = cod;
  }
  public int getCodiceInserito() {
      return codiceInserito;
  }

  public String getContoBancario() {
     if (codiceInserito == codice) {
        return contoBancario;
     } else {
       return "codice errato!!!";
     }
  }
 //...
}
```

Incapsulamento funzionale

È stato analizzato l'incapsulamento all'interno delle classi dove venivano nascosti gli attributi mediante il modificatore private. Nulla vieta di utilizzare private, anche come modificatore di metodi, ottenendo così un **incapsulamento funzionale.**

Incapsulamento funzionale

Un metodo privato, infatti, potrà essere invocato solo da un metodo definito nella stessa classe, che potrebbe a sua volta essere dichiarato pubblico.

Incapsulamento funzionale

Considerando il Class Diagram si avrà:

ContoBancario
-contoBancario: String = "50000 euro" -codice: int = 1234 -codiceInserito: int
+setCodiceInserito(codiceDaTestare:int): void +getCodiceInserito(): int +getContoBancario(): int -controllaCodice(codiceDaTestare:int): String

Incapsulamento funzionale

Considerando il codice:

```
package contobancario;

public class ContoBancario {

    //...

   public String getContoBancario(int codiceDaTestare) {
      return controllaCodice(codiceDaTestare);
   }

   private String controllaCodice(int codiceDaTestare) {
      if (codiceInserito == codiceDaTestare) {
         return contoBancario;
      }
      else {
         return "codice errato!!!";
      }
   }
}
```

Incapsulamento e riuso

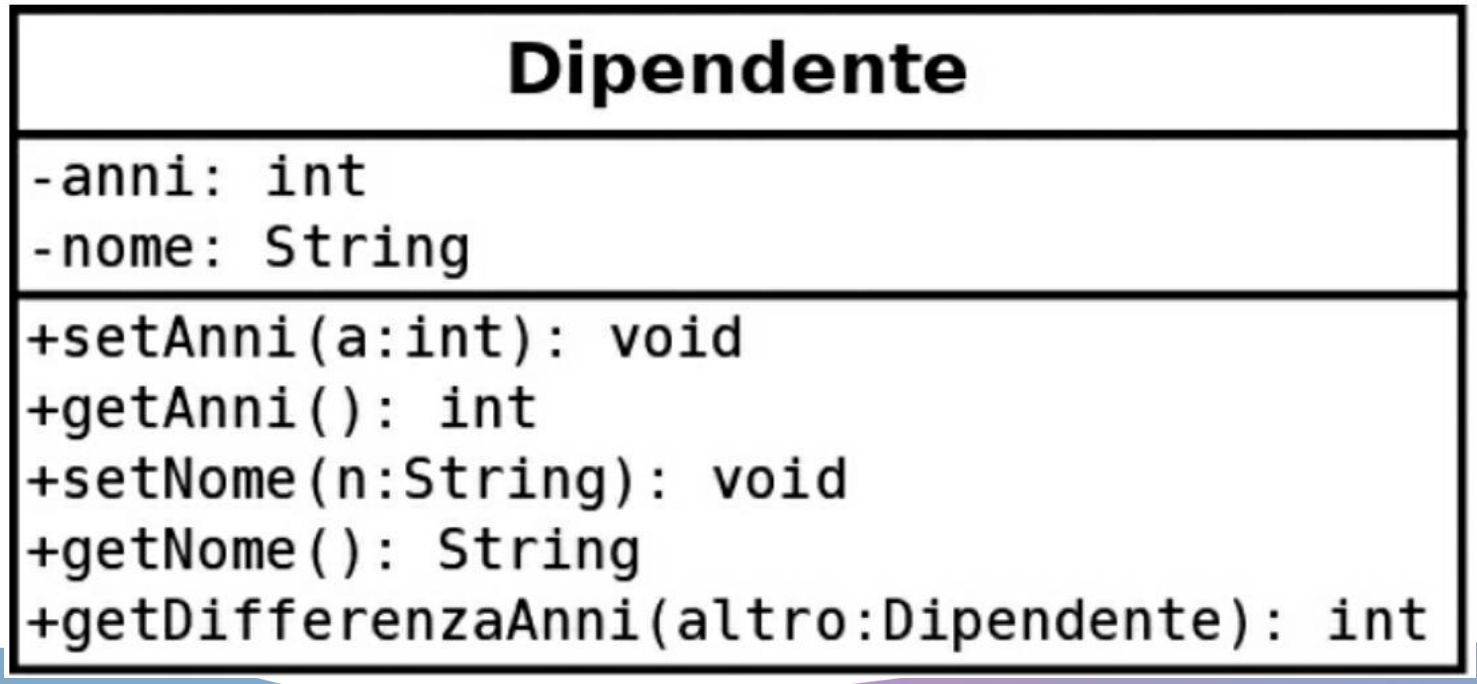
Solitamente si pensa che un membro di una classe dichiarato private diventi “inaccessibile da altre classi”.

Incapsulamento e riuso

L’incapsulamento infatti permetterà a due oggetti istanziati dalla stessa classe di accedere in “modo pubblico” ai rispettivi membri privati.

Incapsulamento e riuso

Si consideri la classe Dipendente rappresentata in UML:



Incapsulamento e riuso

L'implementazione in Java potrebbe essere la seguente:

```
package dipendente;
public class Dipendente {
   private String nome;
   private int anni; // intendiamo età in anni
   //...
   public String getNome() { return nome; }
   public void setNome(String n) { nome = n; }
   public int getAnni() { return anni; }
   public void setAnni(int n) { anni = n; }
   public int getDifferenzaAnni(Dipendente altro) {
      return (anni - altro.anni);
   }
}
```

Incapsulamento e riuso

Nel metodo getDifferenzaAnni() notiamo che è possibile accedere direttamente alla variabile privata ***anni*** dell'oggetto ***altro***, senza dover utilizzare il metodo getAnni().

Incapsulamento e riuso

Modificando il metodo getDifferenzaAnni() nel seguente modo:

```
public int getDifferenzaAnni(Dipendente altro) {

    return (getAnni() - altro.getAnni());

}
```

Favorirebbe il riuso del codice.

Il reference *this*

Java introduce una parola chiave che per definizione coincide con il **reference** all’oggetto corrente: **this**, cioè l'istanza specifica alla quale appartiene quel costruttore/metodo invocato.

Il reference *this*

Si ipotizzi di istanziare due particolari oggetti, aCompleanno e bCompleanno, entrambi hanno una propria variabile giorno.

Il reference *this*

Eseguendo la seguente istruzione:

System.out.println(aCompleanno.getGiorno());

verrà stampato a video il valore della variabile giorno dell'oggetto aCompleanno.

Il reference *this*

Quindi il metodo getGiorno() che è definito dal seguente codice:

```
public int getGiorno() {
    return giorno;
}
```

deve usare un reference in qualche modo per la variabile giorno.

Uso di this

In ogni corpo di un metodo d'istanza (o di un costruttore) è sempre disponibile la variabile **this**:

È un riferimento all'oggetto su cui si invoca il metodo (o costruttore).

È detto anche parametro implicito del metodo.

Uso di this

Può essere usato per accedere alle variabili di istanza del metodo.
Si confronti lo stesso codice con e senza **this**.

```
package punto;

public class Punto {

  public double x , y;

  public String toString () {
      return "(" + x + "," + y + ")";
  }
}
```

Uso di this

```
package punto;

public class Punto {

  public double x , y;

  public String toString () {
      return "(" + x + "," + y + ")";
 }
}
```

Uso di this

L’uso di **this** è indispensabile quando ci si trova a dover distinguere tra un attributo e una variabile con lo stesso nome, come avviene spesso nei metodi setter e nei costruttori.

Uso di this

Si ricordi che esistono le variabili locali e le variabili d’istanza ciò consente al compilatore di dichiarare nella stessa classe, una variabile locale (o un parametro di un metodo) ed una variabile d’istanza con lo stesso identificatore.

Uso di this

JVM alloca le variabili locali e le variabili d’istanza in differenti aree di memoria, dette rispettivamente **Stack Memory e Heap Memory.**

Uso di this

Nella **memoria Stack** vengono memorizzati temporaneamente i dati definiti da un metodo in esecuzione, e una volta terminato il metodo tali dati saranno rimossi dalla memoria.

Uso di this

Nella **Heap Memory** vengono invece memorizzati gli oggetti e i propri dati, e questi saranno deallocati dalla Garbage Collection automaticamente quando questa si assicurerà che non siano più utilizzabili dal programma.

Uso di this

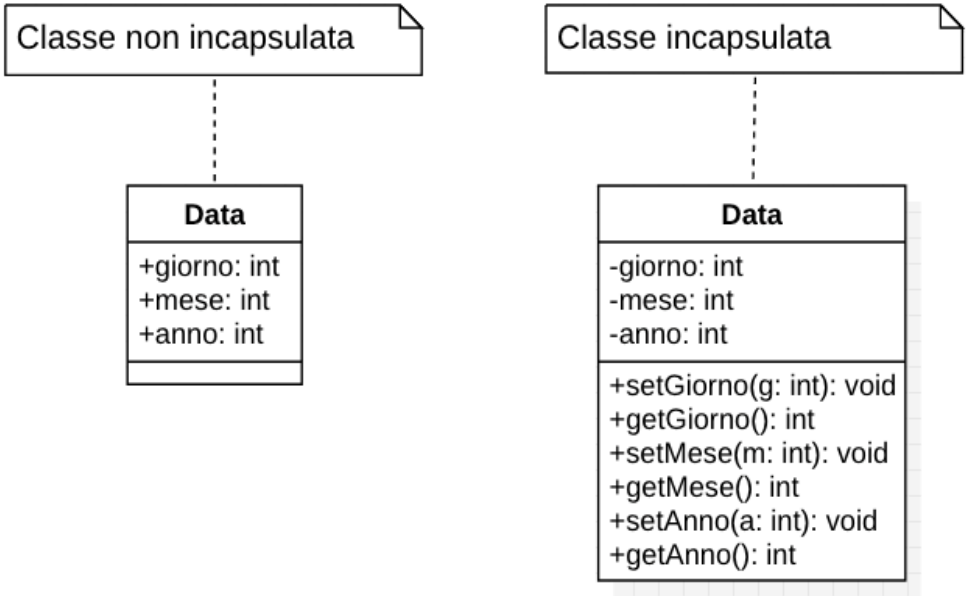
La parola chiave **this** si inserisce in questo discorso specificando che la variabile referenziata appartiene all’istanza.

Quando utilizzare l’incapsulamento

Quando si crea una classe pubblica con attributi privati e tramite i metodi **get** e **set** è possibile interagire rispettivamente prendendo e istanziando dati, al di fuori della classe.

Quando utilizzare l’incapsulamento

Quindi si parla di incapsulamento quando si ha una classe con attributi privati e con i metodi **getter e setter.**



Quando utilizzare l’incapsulamento

Esempio pratico

Si crei un nuovo progetto:

- Una public class Implementazione
- Una public class Anagrafica

Tutto all’interno del **package** implementazione

Quando utilizzare l'incapsulamento

Rispettivamente

```
package implementazione;
public class Anagrafica {
    private String nome;
    private int annoDiNascita;
    public String getNome() {
    return nome;
    }
    public void setNome(String nome) {
    this.nome = nome;
    }
    public int getAnnoDiNascita() {
    return annoDiNascita;
    }
    public void setAnnoDiNascita(int annoDiNascita) {
    this.annoDiNascita = annoDiNascita;
    }
    //prendo l'oggetto e lo trasformo in una stringa
    @Override
    public String toString() {
        return "Anagrafica [nome=" + nome + ", anno di nascita=" + annoDiNascita + "]";
    }
}
```

Sono state dichiarate le variabili che si utilizzeranno

Metodi get e set per operare sulle variabili

Definisco il «modo di visualizzazione» quando richiamo l'oggetto anagrafica

Quando utilizzare l'incapsulamento

```
package implementazione;

public class Implementazione {

    public static void main(String[] args) {
        Anagrafica anagrafica = new Anagrafica();

        Anagrafica anagrafica2 = new Anagrafica();

        anagrafica.setNome("Francesco");

        anagrafica.setAnnoDiNascita(1990);

        anagrafica2.setNome("Paola");

        anagrafica2.setAnnoDiNascita(1987);

        System.out.println("Nome="+anagrafica.getNome()+"\nNome="+anagrafica2.getNome());
        System.out.println(anagrafica);
        System.out.println(anagrafica2);
    }
}
```

Sto istanziando un nuovo oggetto anagrafica

Modifica dei valori delle variabili

Quando utilizzare l’incapsulamento

Quando si crea un package deve avere lo stesso nome della cartella che contiene i file, nello specifico implementazione, ma in minuscolo.

Quando utilizzare l’incapsulamento

Eseguendo il codice

Markers Properties Servers Snippets Console

```
<terminated> Implementazione [Java Application] /Library/Java/JavaVirtualMachines/jdk-12.0.1.jdk/Contents/Home/bin/java (25 lug 2019, 10:07:13)
Nome = Francesco
Nome = Paola
Anagrafica [nome=Francesco, anno di nascita=1990]
Anagrafica [nome=Paola, anno di nascita=1987]
```

Gestione dei package

Il significato di **public** è chiaro quando applicato a variabili e metodi. Grazie al concetto che la libreria standard di Java sia organizzata in package, il programmatore ha la possibilità di organizzare anche le proprie classi.

Gestione dei package

È molto semplice infatti dichiarare una classe appartenente ad un package.

L'istruzione package deve essere obbligatoriamente la prima in un file Java!

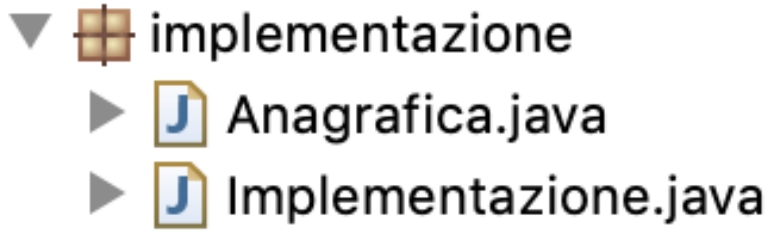
Gestione dei package

La parola chiave **package** permette di specificare, prima della dichiarazione della classe, il package di appartenenza.

Gestione dei package

I package fisicamente sono semplici cartelle (directory).

Considerando il codice circa il package implementazione, dopo aver dichiarato la classe appartenente a questo package, bisogna inserire a sua volta il tutto all'interno di una cartella chiamata implementazione.



Gestione manuale

Se ad esempio si utilizza il Blocco Note per usare i package, è consigliabile seguire i seguenti step.

Quando si inizia un progetto, è opportuno creare una cartella con il nome del progetto stesso (P.E. Bancomat).

Gestione manuale

La cartella dovrà contenere almeno due sottocartelle:

Src: dove vanno inseriti tutti i sorgenti.

Classes: dove si troveranno tutti i file compilati.

Gestione manuale

In questo modo l’ambiente di sviluppo è diviso dall’ambiente di distribuzione.

È consigliato creare altre cartelle:

Docs: dove mettere tutta la documentazione.

Config: dove mettere i file di configurazione.

Gestione manuale

Per esempio, nella cartella Bancomat inserire le cartelle src, classes, etc. Ogni volta che una classe deve appartenere ad un package, creare la cartella-package ed inserire il file sorgente all’interno.

Gestione manuale

Naturalmente è possibile creare sia le cartelle manualmente o tramite il terminale utilizzando il comando **mkdir.**

Gestione manuale

Per esempio, se la classe ContoCorrente deve appartenere al package banca, creare la cartella banca all’interno della cartella src e inserire il file ContoCorrente.java nella cartella banca.

Gestione manuale

Ogni volta che bisogna compilare, bisogna sempre posizionarsi tramite il prompt di DOS nella cartella classes e da qui eseguire un comando del tipo:

javac –d ...\src\nomepackage*.java

Se invece si desidera compilare tutte le classi del package banca, bisogna eseguire il seguente comando:

javac –d ...\src\banca*.java

Moduli

Un modulo Java è uno o più pacchetti Java.

Il nome di un modulo Java segue le stesse regole di denominazione dei pacchetti Java.

Modificatori d’accesso

Esistono diversi modificatori, tra cui **final, static, public e private**. Questi ultimi vengono definiti come modificatori di accesso (o anche specificatori d’accesso) perché regolano essenzialmente la visibilità e l’accesso ad un componente Java.

Modificatori d’accesso

Il modificatore **public** può essere applicato sia ad un membro (attributo o metodo) di una classe, sia ad una classe stessa.

Modificatori d’accesso

Un membro dichiarato pubblico sarà accessibile da una qualsiasi classe situata in qualsiasi package, e che per l’incapsulamento le variabili d’istanza non dovrebbero mai essere dichiarate pubbliche.

Modificatori d’accesso

Una **classe dichiarata pubblica** sarà anch’essa visibile da un qualsiasi package (a meno di dichiarazione di moduli).

Modificatori d’accesso

Accesso pubblico.

Il modificatore di accesso **public** specifica che la classe è accessibile da qualunque parte del programma.

Accesso a livello di package.

L’assenza del modificatore di accesso public specifica che la classe è accessibile solo all’interno del package.

Modificatori d’accesso

In questo caso è buona norma:

Definire private i campi membro.

Dare alle funzioni membro l’accessibilità a livello di package.

Non è possibile dichiarare private o protected una classe.

Modificatori d’accesso

Il modificatore **private** restringe la visibilità di un membro di una classe alla classe stessa, e non è applicabile alla dichiarazione di una classe.

Nessun modificatore d’accesso

È possibile **evitare di usare modificatori** sia relativamente ad un membro (attributo o metodo) di una classe, sia relativamente ad una classe stessa.

Modificatore protected

Il modificatore d’accesso **protected** definisce per un membro il grado più accessibile dopo quello definito da public.

Nessun modificatore d’accesso

La differenza tra membri dichiarati senza modificatore d’accesso e membri dichiarati con il modificatore protected è che i membri protetti saranno ereditati come fossero pubblici anche nelle sottoclassi esterne al package di dichiarazione.

Nessun modificatore d’accesso

Anche se non si antepone il public, la classe stessa sarà visibile solo dalle classi appartenenti allo stesso package. In questo caso si parla di visibilità di package.

Ricapitolando

Ricapitolando i diversi modificatori:

MODIFICATORE	STESSA CLASSE	STESSO PACKAGE	SOTTOCLASSE	OVUNQUE
`public`	SI	SI	SI	SI
`protected`	SI	SI	SI	NO
nessun modificatore	SI	SI	NO	NO
`private`	SI	NO	NO	NO

Ricapitolando

Caratteristica	Classe	Attributo	Metodo
public	x	x	x
protected		x	x
private		x	x
static		x	x
abstrac	x		x
final	x	x	x

Il modificatore static

Il modificatore **static** utilizzato per marcare i metodi e gli attributi di una classe (quindi tutti i membri di una classe) ma anche per gli inizializzatori.

Il modificatore static

Una classe può cioè definire dei metodi che essa stessa (non un suo oggetto) eseguirà.

Tali metodi si chiamano **metodi statici, o anche metodi di classe**
cioè un metodo statico va invocato sulla classe.

Il modificatore static

È possibile incapsulare una variabile statica, ma quella variabile verrà comunque condivisa da tutti gli oggetti istanziati dalla stessa classe quindi in generale l'incapsulamento non si applica a variabili statiche.

Il modificatore static

Per quanto detto, un membro statico ha la caratteristica di poter essere utilizzato mediante una sintassi del tipo:

NomeClasse.nomeMembro

in luogo di:

nomeOggetto.nomeMembro

Il modificatore static

Un metodo statico è un metodo che è condiviso da tutti gli oggetti istanziati dalla classe in cui è dichiarato.

Un esempio di metodo statico è il metodo sqrt().

```
Math.sqrt(numero)
```

Il modificatore static

Math è il nome della classe e non il nome di un’istanza di quella classe.

Il modificatore static

Posso definire un metodo statico, static, all'interno di una classe per poterlo poi utilizzare con la seguente sintassi:

Classe.NomeMetodoStatico

```
class NewPoint {
static int somma(int a, int b) {
return a+b; }
...
```

```
x=NewPoint.somma(12,43);
```

Ricapitolando

Metodo non statico: è una operazione che faccio con un oggetto (posso usare this) metodo statico: è un metodo che è associato alla classe.

Static import

Grazie ad **import** è possibile importare all'interno dei nostri file i nomi delle classi e delle interfacce, in modo tale che si possano poi usare senza dover specificare l'intero fully qualified name.

Domande e approfondimenti